

PYTHON

Module 10: Type Casting

CODING

The Concept of Casting

In Python, casting is the process of converting a variable from one data type to another. Since Python is object-oriented, it uses classes/constructor functions to perform these conversions.

Casting is essential when:

- Receiving user input (which is always a string by default).
- Performing math on numbers stored as strings.
- Formatting numeric data for display in text reports.

Converting to Integer: int()

The `int()` function constructs an integer from various sources.

Rules/Behavior:

- From Float: It **truncates** the decimal (35.88 becomes 35). It does not round!

- From String: Only works if the string represents a whole number (e.g., `"10"`).

Example:

```
int(2.8) results in 2
```

```
int("100") results in 100
```

Converting to Float: float()

The `float()` function constructs a floating-point number.

Rules/Behavior:

- From Integer: Adds a trailing `.0` (e.g., 5 becomes 5.0).

- From String: Works for strings representing both whole and decimal numbers.

Example:

```
float(1) results in 1.0
```

```
float("3.14") results in 3.14
```

Converting to String: str()

The `str()` function is the most flexible casting tool. It can convert almost any Python object into a string representation.

Rules/Behavior:

- Use this when you need to concatenate numbers with text using the `+` operator.

Example:

`str(10)` results in `'10'`

`str(10.5)` results in `'10.5'`

Implicit vs Explicit Casting

• **Explicit Casting:** When you manually use functions like `int()` or `float()`. This is what we focus on in this module.

• **Implicit Casting:** When Python automatically converts a type (e.g., adding an `int` to a `float` results in a `float` automatically).

At CodeHive, we recommend ****Explicit Casting**** for clarity so other developers know exactly what data type to expect.

Common Errors in Casting

Conversion doesn't always work. You must ensure the source value is compatible with the target type.

The `ValueError: Attempting`
`int("Hello")` or `int("4.2")` will result in a `ValueError` because the characters aren't valid whole numbers.

Tip: Always wrap casting in a validation check when dealing with user-generated data.

Knowledge Check

Test your logic:

Question: What will be the result of
`print(int(35.88))` ?

****Answer: 35****

Why? Because `int()` removes all decimals without
rounding up or down.